Tecnológico de Estudios Superiores de Ecatepec
Ingeniería en Sistemas Computacionales

PARCIAL 2 · PROYECTO INTEGRADOR

Smart Academic Advisor

Núcleo funcional: StudentAnalyzerFunction y dashboard académico

Documentación del LINQ

Programación Lógica y Funcional · 7.° semestre

Equipo 6
Profesora Alejandra González Sánchez
Grupo 5701
Fecha 30 de octubre de 2026
Integrantes Hernández López Daniel Javier
Pérez González Jean Carlo (líder)
Robles Roldán Rosalba
Sanchez Soto Dana Amelie
Valadez Rivera Héctor Gabriel

Índice

1. Objetivo y alcance....3
2. Arquitectura de la solución....3
3. Datos de entrada....4
4. Catálogo de consultas LINQ....4
5. Consultas principales explicadas....5
6. Funciones puras y evaluación de expresiones....7
7. Indicadores y recomendaciones (análisis y síntesis)....7
8. StudentAnalyzerFunction: contrato HTTP....8
9. Dashboard y gestión de alumnos, materias y especialidades....10
10. Pruebas y verificación....14
11. Despliegue en Azure....14
12. Conclusiones....15

1. Objetivo y alcance

La segunda parte del proyecto construye el núcleo funcional del Smart Academic Advisor: el dashboard académico, la gestión de alumnos, materias y especialidades, y las estadísticas académicas. Las colecciones JSON se procesan con LINQ (Select, Where, GroupBy, OrderBy y Aggregate) desde la StudentAnalyzerFunction, una Azure Function HTTP.

El historial de cada alumno se sintetiza en indicadores (promedios por área, avance e intereses) y en recomendaciones básicas, usando funciones puras, listas inmutables y evaluación de expresiones propias del paradigma funcional. Este documento explica cada consulta LINQ, qué hace y qué resultado produce con los datos reales del proyecto.

Entregable	Dónde está
Dashboard académico operativo	SmartAdvisor.Web (ASP.NET Core MVC)
StudentAnalyzerFunction con LINQ	SmartAdvisor.Functions + SmartAdvisor.Core/Services/StudentAnalyzer.cs
LINQ documentado	Este documento (secciones 4 y 5) y comentarios [LINQ …] en el código
Sitio desplegado	Guía y script en la sección 11 (desplegar-azure.ps1)

2. Arquitectura de la solución

La lógica vive en una biblioteca (Core) sin dependencias de Azure ni de ASP.NET. Así la misma función pura sirve a la Azure Function, al sitio web (como respaldo) y a las pruebas.

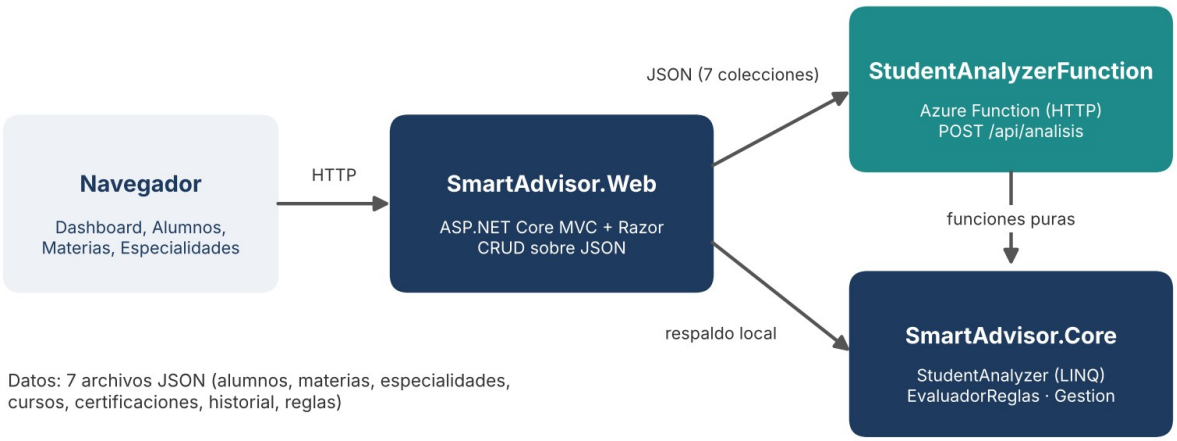


Figura 1. El sitio llama a la Function; si no responde, calcula localmente con el mismo Core.

Proyecto	Responsabilidad
SmartAdvisor.Core	Modelos inmutables (records), StudentAnalyzer (LINQ), EvaluadorReglas, Gestion (validaciones y listas), RepositorioJson.

Proyecto	Responsabilidad
SmartAdvisor.Functions	Azure Functions (modelo aislado, .NET 8): Health, StudentAnalyzerFunction y AnalisisDeAlumno.
SmartAdvisor.Web	Dashboard y CRUD en MVC + Razor. Llama a la Function por HTTP y tiene respaldo local.
SmartAdvisor.Tests	24 pruebas xUnit sobre el análisis, las reglas y la gestión.

3. Datos de entrada

Las colecciones están en archivos JSON validados con JSON Schema (carpeta schemas). El StudentAnalyzer recibe las siete juntas como un solo objeto DatosAcademicos.

Colección	Registros	Contenido
alumnos	4	Matrícula, semestre, promedio, intereses y estatus.
materias	10	Área (IA, Desarrollo, Datos…), créditos, semestre y prerrequisitos.
especialidades	3	Áreas clave, promedio mínimo, materias, cursos y certificaciones.
cursos	3	Cursos recomendados por especialidad.
certificaciones	3	Certificaciones recomendadas por especialidad.
historial	10	Materia cursada por alumno, estatus (aprobada, reprobada, cursando) y calificación.
reglas	4	Condiciones (Promedio, Interés) que concluyen una especialidad.

4. Catálogo de consultas LINQ

Cada método de StudentAnalyzer lleva una etiqueta [LINQ …] en su comentario. La tabla resume todas las consultas; el resultado es el que produce el sistema con los datos de ejemplo.

#	Operador(es)	Método	Qué hace	Resultado real
1	Where	Historial(historial, alumnoId)	Filtra el historial de un alumno.	Ana: 5 materias aprobadas
2	Where	Aprobadas(historial)	Deja solo las materias con estatus aprobada.	—
3	GroupBy + Join + Average	IndicadoresPorArea	Agrupa las materias del plan por área, las cruza con las aprobadas y promedia.	IA 93.0 · Desarrollo 88.8 · Datos 83.7
4	Join + Aggregate	CreditosAprobados	Suma créditos con semilla 0 (acumulador).	Ana: 25 de 50 créditos (50 %)
5	Aggregate	EstadisticaDe	Calcula n, mínimo, máximo y suma en una sola pasada.	n=4 · mín 70.5 · máx 95.0 · suma 326.0

#	Operador(es)	Método	Qué hace	Resultado real
6	SelectMany + GroupBy + OrderBy	InteresesFrecuentes	Aplana los intereses, los cuenta y ordena de mayor a menor.	Datos 2; los demás 1
7	GroupBy	PorEstatus	Cuenta alumnos por estatus.	activo: 4
8	OrderByDescending + Take	MejoresPromedios	Los tres mejores promedios.	Ana 95.0 · Luis 82.5 · Sofía 78.0
9	Join + Where + Average	Afinidad	Promedio de las materias aprobadas del área clave de la especialidad.	Ana, IA: 93.0
10	Select + OrderByDescending	Recomendar (ranking)	Calcula la afinidad con cada especialidad y las ordena.	Ana: IA 96.3 · Datos 57.0 · Desarrollo 57.0
11	Join + Where + All + OrderBy	MateriasSugeridas	Materias de la especialidad no aprobadas ni cursadas con prerrequisitos cumplidos.	Ana: Aprendizaje Automático y Visión Computacional
12	Select + Count + Any	Analizar (resumen)	Totales y conteos por especialidad.	Datos: 2 interesados, 1 condicionado

Operadores cubiertos de los que pide la maestra: Select, Where, GroupBy, OrderBy y Aggregate. Además se usan Join, SelectMany, Take, Average, Sum, Count, Any, All, Distinct y ToHashSet.

5. Consultas principales explicadas

5.1 Promedio y avance por área (GroupBy + Join + Average)

Parte de las materias del plan y las agrupa por área. Cruza con las aprobadas mediante Join, promedia las calificaciones y calcula el avance como aprobadas entre las posibles.

```
var porMateria = aprobadas
    .Join(materias, h => h.MateriaId, m => m.Id,
          (h, m) => (m.Area, Nota: h.Calificacion!.Value))
    .ToList();

return materias
    .GroupBy(m => m.Area)
    .Select(g =>
    {
        var notas = porMateria.Where(t => t.Area == g.Key)
                              .Select(t => t.Nota).ToList();
        var posibles = g.Count() * Math.Max(alumnos, 1);
        return new IndicadorArea(g.Key,
            notas.Count == 0 ? null : Redondear(notas.Average()),
            notas.Count, g.Count(),
            Redondear(100m * notas.Count / posibles));
    })
    .OrderBy(i => i.Area).ToList();
```

Área	Promedio	Aprobadas	Materias del plan	Avance
Datos	83.7	3	3	25.0 %
Desarrollo	88.8	4	3	33.3 %
IA	93.0	2	4	12.5 %

El avance global considera a los 4 alumnos: Datos tiene 3 aprobadas de 3 × 4 = 12 posibles (25 %).

5.2 Créditos y estadística (Aggregate)

Aggregate recorre la colección acumulando un resultado. En los créditos el acumulador es un entero con semilla 0; en la estadística es un record inmutable que devuelve uno nuevo en cada paso.

```
public static int CreditosAprobados(IEnumerable<Materia> materias,
                                    IEnumerable<Historial> aprobadas) =>
    aprobadas
        .Join(materias, h => h.MateriaId, m => m.Id, (_, m) => m.Creditos)
        .Aggregate(0, (acumulado, creditos) => acumulado + creditos);

public static Estadistica EstadisticaDe(IEnumerable<decimal> valores) =>
    valores.Aggregate(Estadistica.Vacia, (acc, x) => acc.Agregar(x));
```

5.3 Intereses más frecuentes (SelectMany + GroupBy + OrderBy)

```
alumnos
    .SelectMany(a => a.Intereses.Distinct(StringComparer.OrdinalIgnoreCase))
    .GroupBy(i => i, StringComparer.OrdinalIgnoreCase)
    .Select(g => new ConteoInteres(g.Key, g.Count()))
    .OrderByDescending(c => c.Alumnos).ThenBy(c => c.Interes)
    .ToList();
```

SelectMany convierte la lista de intereses de cada alumno en una sola secuencia. Resultado: Datos 2; Desarrollo, IA, Redes y Seguridad 1 cada uno.

5.4 Mejores promedios (OrderByDescending + Take)

```
alumnos.OrderByDescending(a => a.Promedio).ThenBy(a => a.Nombre)
       .Take(cuantos)
       .Select(a => new MejorPromedio(a.Id, a.Nombre, a.Semestre,
                                      a.Promedio, a.Intereses.FirstOrDefault() ?? "—"))
       .ToList();
```

5.5 Materias sugeridas (Join + Where + All + OrderBy)

Sugiere las materias de la especialidad recomendada que el alumno todavía no aprueba ni cursa y cuyos prerrequisitos ya aprobó.

```
return esp.MateriasIds
    .Join(materias, id => id, m => m.Id, (_, m) => m)
    .Where(m => !aprobadas.Contains(m.Id) && !cursando.Contains(m.Id))
    .Where(m => m.Prerrequisitos.All(aprobadas.Contains))
    .OrderBy(m => m.Semestre).ThenBy(m => m.Nombre)
    .Select(m => new MateriaSugerida(/* … */))
    .ToList();
```

6. Funciones puras y evaluación de expresiones

6.1 Funciones puras

Todos los métodos de StudentAnalyzer, EvaluadorReglas y Gestion son estáticos y puros: el resultado depende solo de los argumentos, no leen ni escriben estado global y no modifican las colecciones recibidas. Una prueba lo comprueba: se analiza dos veces el mismo DatosAcademicos y el resultado es idéntico y los datos de entrada no cambian.

6.2 Listas inmutables

Los modelos son records de solo lectura y las colecciones son IReadOnlyList. Agregar, reemplazar o quitar devuelve una lista nueva en lugar de mutar la original.

```
public static IReadOnlyList<T> Agregar<T>(IEnumerable<T> lista, T elemento) =>
    lista.Append(elemento).ToList();
public static IReadOnlyList<T> Quitar<T>(IEnumerable<T> lista, Func<T, bool> predicado) =>
    lista.Where(x => !predicado(x)).ToList();
```

6.3 Evaluación de expresiones

Cada condición de reglas.json, por ejemplo { "hecho": "Promedio", "operador": ">=", "valor": 85 }, se compila a una función Hechos → bool. El operador de texto se traduce a una lambda (función de orden superior) y las condiciones de una regla se componen con Aggregate usando «Y».

```
public static Func<decimal, decimal, bool> Comparador(string operador) => operador switch
{
    ">=" => (a, b) => a >= b,   ">" => (a, b) => a > b,
    "<=" => (a, b) => a <= b,   "<" => (a, b) => a < b,
    "="  => (a, b) => a == b,   "!=" => (a, b) => a != b,
    _    => (_, _) => false
};

public static Func<Hechos, bool> Compilar(Regla regla) =>
    regla.Condiciones
         .Select(Compilar)
         .Aggregate(Siempre, (acumulada, siguiente) => h => acumulada(h) && siguiente(h));
```

Un hecho que el evaluador no conoce (por ejemplo los derivados de REG-004) hace falsa la condición en lugar de lanzar un error.

7. Indicadores y recomendaciones (análisis y síntesis)

La síntesis convierte el historial en pocos indicadores legibles y en una recomendación explicada. Las reglas filtran las especialidades posibles y la afinidad las ordena.

$$\text{Afinidad}(e) = 100 \times (0.40 \cdot I + 0.35 \cdot H + 0.25 \cdot P)$$

- **I** proporción de intereses del alumno que coinciden con las áreas clave de la especialidad.
- **H** promedio de sus materias aprobadas de esas áreas, entre 100 (0 si no tiene).
- **P** promedio general entre 100.

Ejemplo, Ana y la especialidad IA: I = 1.00 (su único interés es IA), H = 0.93 (promedio de sus materias de IA) y P = 0.95. Afinidad = 100 × (0.40·1.00 + 0.35·0.93 + 0.25·0.95) = 96.3.

Alumno	Promedio	Estado	Especialidad	Afinidad	Motivo
Ana Torres	95.0	recomendada	Inteligencia Artificial	96.3	Cumple REG-001 (≥ 85 e interés IA).
Luis Mendoza	82.5	recomendada	Desarrollo de Software	69.5	Cumple REG-002 y REG-003; gana la de mayor afinidad.
Sofía Ramírez	78.0	condicionada	Ciencia de Datos	86.8	Le faltan 2.0 puntos de promedio (mínimo 80).
Diego Herrera	70.5	sin recomendación	—	17.6	Sus intereses no coinciden con ninguna especialidad.

Estados posibles: «recomendada» (cumple una regla), «condicionada» (hay interés pero falta cumplir la regla) y «sin-recomendacion». Cada recomendación incluye el ranking completo para que se pueda explicar la decisión.

Análisis y síntesis: de los datos al indicador

Datos de entrada (análisis)	Indicador (síntesis)	Decisión
Calificaciones del historial por materia	Promedio por área y avance en créditos	Detectar áreas fuertes y débiles
Intereses declarados de cada alumno	Intereses más frecuentes en el grupo	Dimensionar la demanda por especialidad
Promedio, intereses y reglas	Afinidad y estado de la recomendación	Recomendar o explicar qué falta
Prerrequisitos y avance del alumno	Materias sugeridas	Indicar el siguiente paso concreto

8. StudentAnalyzerFunction: contrato HTTP

StudentAnalyzerFunction es una Azure Function HTTP (modelo aislado, .NET 8, trigger anónimo). Devuelve JSON en camelCase.

Función	Ruta	Descripción
Health	`GET /api/health`	Comprueba que la Function está viva.
StudentAnalyzerFunction	`POST /api/analisis`	Recibe un DatosAcademicos y devuelve el AnalisisAcademico completo. 400 si el cuerpo es nulo o JSON inválido.
StudentAnalyzerFunction	`GET /api/analisis`	Analiza los datos JSON incluidos en la Function.
AnalisisDeAlumno	`GET /api/alumnos/{id}/analisis`	Análisis de un alumno. 404 si no existe.

Estructura de la respuesta (resumen):

```
{
  "resumen": { "totalAlumnos": 4, "totalMaterias": 10, "creditosPlan": 50,
               "promedioGeneral": 81.5, "sinRecomendacion": 1 },
  "estadisticaPromedios": { "n": 4, "min": 70.5, "max": 95.0,
                            "suma": 326.0, "promedio": 81.5 },
  "porArea": [ { "area": "IA", "promedio": 93, "aprobadas": 2,
                 "materiasPlan": 4, "avancePct": 12.5 }, ... ],
  "intereses": [ { "interes": "Datos", "alumnos": 2 }, ... ],
  "mejoresPromedios": [ ... ],
  "especialidades": [ { "nombre": "Ciencia de Datos",
                        "alumnosInteresados": 2, "condicionados": 1 }, ... ],
  "alumnos": [ { "nombre": "Ana Torres Ramirez", "avancePct": 50,
                 "recomendacion": { "especialidad": "Inteligencia Artificial",
                                    "afinidad": 96.3, "estado": "recomendada" } }, ... ]
}
```

El sitio web llama a POST /api/analisis con los datos actuales. Si la Function no responde (error de red, tiempo agotado o JSON inválido), calcula con el mismo Core y muestra un aviso. El dashboard indica el origen con el chip «Procesado por: Azure Function» o «Cálculo local».

9. Dashboard y gestión de alumnos, materias y especialidades

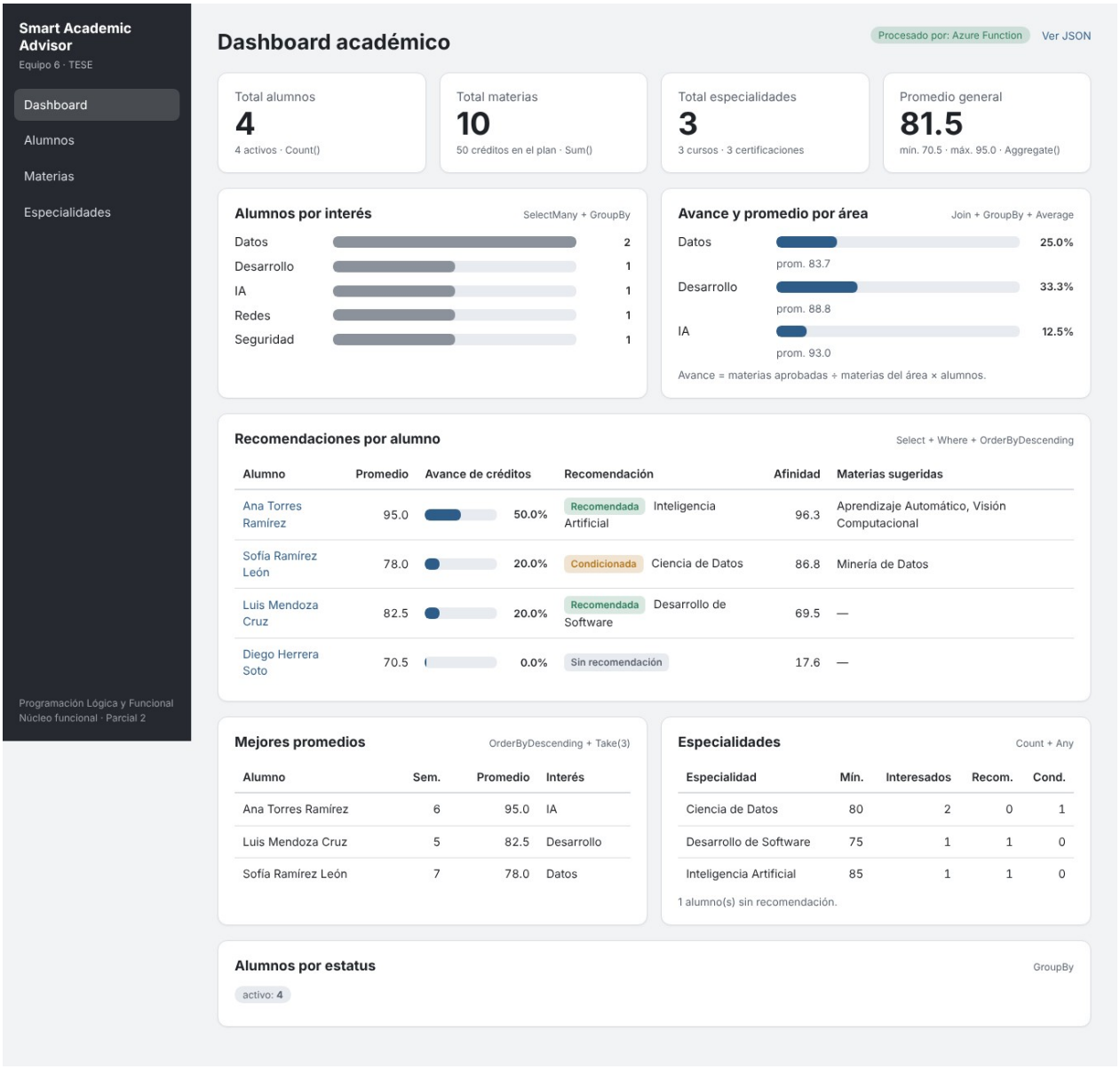


Figura 2. Dashboard: indicadores, barras por área, recomendaciones por alumno, mejores promedios y especialidades.

El dashboard muestra el resumen general, el promedio y avance por área, los intereses frecuentes, las recomendaciones por alumno, los tres mejores promedios y el resumen por especialidad. Tiene tema claro y oscuro.

Gestión de alumnos

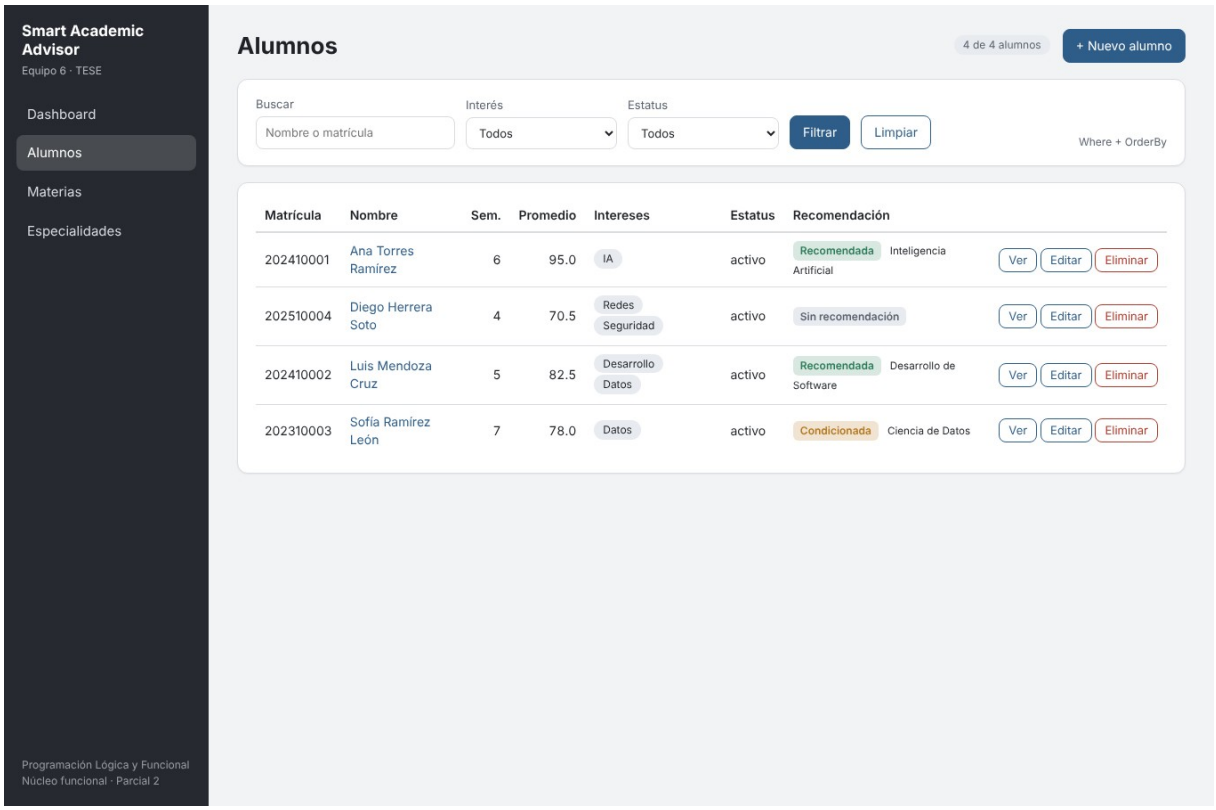


Figura 3. Listado de alumnos con filtros por texto, interés y estatus.

Se puede crear, editar y eliminar alumnos (al eliminar también se borra su historial), registrar y quitar materias del historial y consultar el detalle con la recomendación.

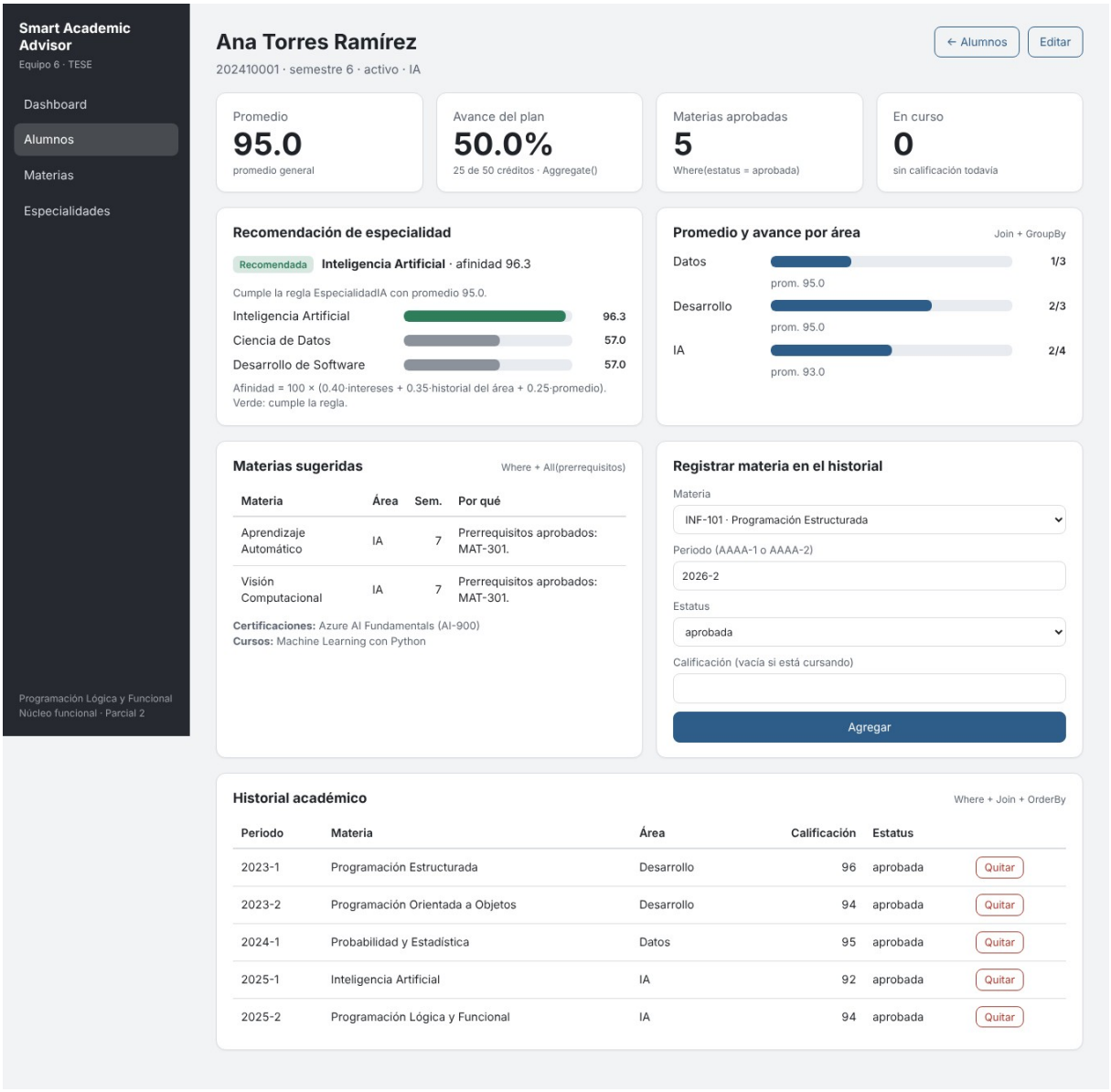


Figura 4. Detalle de un alumno: avance, indicadores por área, recomendación explicada y materias sugeridas.

Gestión de materias y especialidades

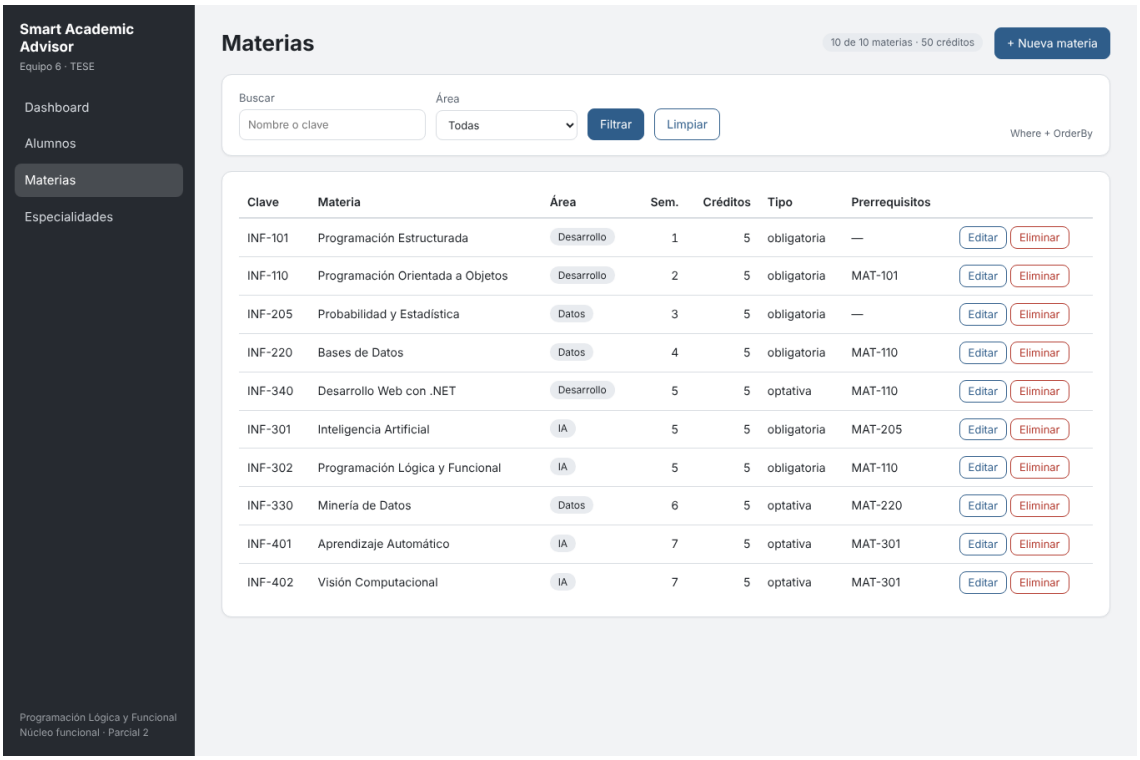


Figura 5. Catálogo de materias.

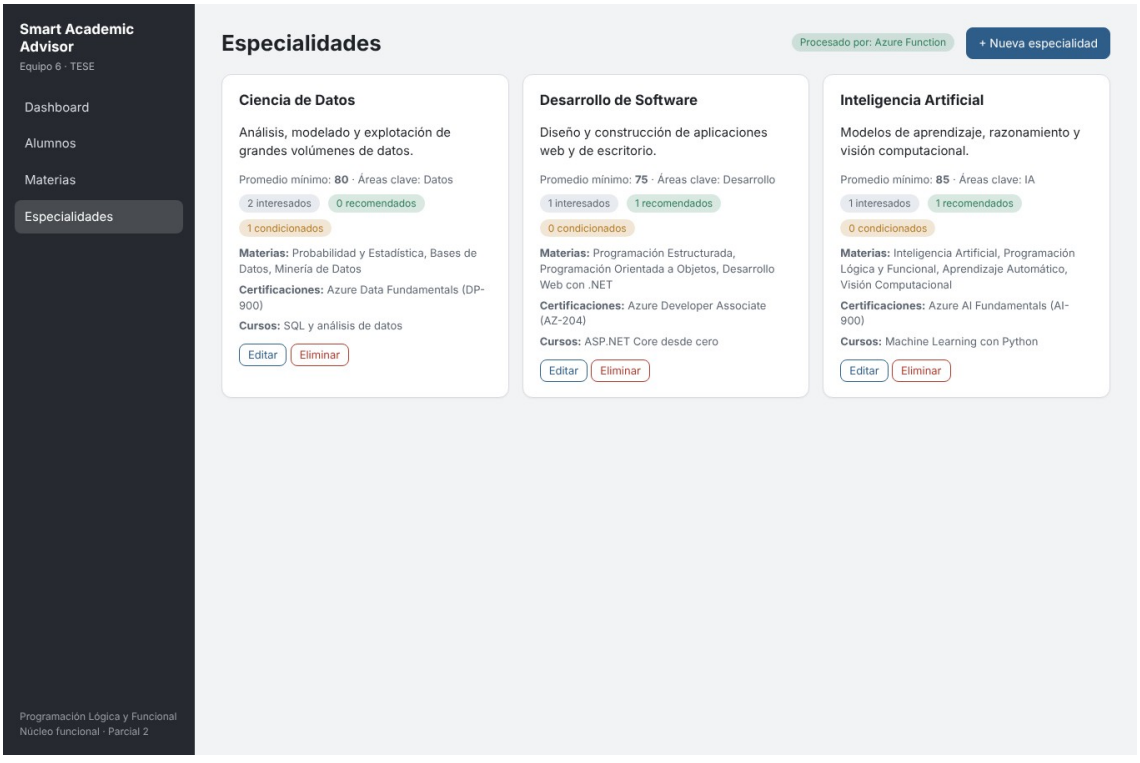


Figura 6. Especialidades con sus áreas clave y promedio mínimo.

Las validaciones impiden datos incorrectos: identificadores duplicados, una materia aprobada con calificación menor a 70, una materia cursando con calificación o eliminar una materia o especialidad de la que otras dependen.

10. Pruebas y verificación

Verificación	Resultado
Pruebas unitarias del análisis, reglas y gestión	24 de 24 correctas
Prueba de punta a punta del CRUD en el navegador (crear, editar, eliminar, historial, validaciones)	17 de 17 correctas
Contrato HTTP de la Function (health, POST y GET /api/analisis, análisis por alumno, 400 y 404)	Correcto con un host de prueba
Dashboard conectado a la Function y respaldo local al detenerla	Ambos modos muestran los mismos valores (promedio 81.5)
Análisis puro: dos ejecuciones iguales y datos de entrada sin cambios	Correcto

Alcance de la verificación. Se compiló y probó con el SDK de .NET 10 en un entorno sin acceso a Azure ni a NuGet. Los proyectos entregados apuntan a net8.0, igual que Series Insights. El despliegue real y la restauración del paquete de Azure Functions deben hacerse desde la computadora del equipo (sección 11).

11. Despliegue en Azure

Requisitos: Azure CLI (az), .NET 8 SDK y Azure Functions Core Tools v4 (func), los mismos de Series Insights. Los nombres de la cuenta de almacenamiento, la Function y el sitio deben ser únicos en todo Azure; si alguno ya existe, cambia el sufijo.

Opción A: un solo script

1. Abre PowerShell y escribe cd «ruta de la carpeta que contiene SmartAdvisor.sln», luego presiona Enter.
2. Si PowerShell bloquea scripts, escribe Set-ExecutionPolicy -Scope Process Bypass y presiona Enter.
3. Escribe .\desplegar-azure.ps1 y presiona Enter. Se abre el navegador para iniciar sesión en Azure.
4. Espera a que termine (varios minutos). Al final abre el sitio. En el dashboard debe verse «Procesado por: Azure Function».

Opción B: comandos paso a paso

```
az login
az group create --name rg-smart-advisor --location northcentralus
az storage account create --name stadvisorjcperez --resource-group rg-smart-advisor `
    --location northcentralus --sku Standard_LRS

# 1) Function
az functionapp create --name func-advisor-jcperez --resource-group rg-smart-advisor `
    --storage-account stadvisorjcperez --consumption-plan-location northcentralus `
    --runtime dotnet-isolated --runtime-version 8 --functions-version 4 --os-type Windows
cd SmartAdvisor.Functions
func azure functionapp publish func-advisor-jcperez
cd ..

# 2) Sitio web
az appservice plan create --name asp-advisor-f1 --resource-group rg-smart-advisor `
    --location northcentralus --sku F1
az webapp create --name web-advisor-jcperez --resource-group rg-smart-advisor `
    --plan asp-advisor-f1 --runtime "dotnet:8"
az webapp config appsettings set --name web-advisor-jcperez --resource-group rg-smart-advisor `
    --settings "StudentAnalyzer__BaseUrl=https://func-advisor-jcperez.azurewebsites.net"
dotnet publish SmartAdvisor.Web -c Release -o publish\web
Compress-Archive -Path publish\web\* -DestinationPath publish\web.zip -Force
az webapp deploy --name web-advisor-jcperez --resource-group rg-smart-advisor `
    --src-path publish\web.zip --type zip
```

Comprobación

1. Abre https://func-advisor-jcperez.azurewebsites.net/api/health: debe responder un JSON con el estado.
2. Abre https://web-advisor-jcperez.azurewebsites.net y revisa el chip «Procesado por: Azure Function» en el dashboard.
3. Toma una captura del sitio y de la Function para el entregable.

Notas. La variable StudentAnalyzer__BaseUrl (con doble guion bajo) le indica al sitio dónde está la Function; si queda vacía, el sitio calcula localmente. Los cambios hechos en el CRUD se guardan en la carpeta %HOME%/smart-advisor-data del servidor, para que no se pierdan al volver a publicar. Estos comandos no se ejecutaron en el entorno de desarrollo por no tener acceso a Azure.

12. Conclusiones

El núcleo funcional queda completo: el StudentAnalyzer transforma siete colecciones JSON en indicadores y recomendaciones con LINQ, el dashboard los presenta y el CRUD mantiene los datos consistentes. Separar la lógica en un Core de funciones puras permitió reutilizarla en la Function, el sitio y las pruebas, y comprobar cada cifra con datos reales.

Aggregate resultó el operador más versátil: sirvió para sumar créditos, calcular estadísticas en una pasada y componer las condiciones de una regla en una sola función. Esa última idea, tratar las reglas como datos que se compilan a funciones, es la que conecta LINQ con el paradigma funcional.

Competencia blanda, análisis y síntesis: el equipo partió del historial en bruto y lo redujo a pocos indicadores y una recomendación explicada, cada uno con su fuente de datos y su regla.